
Distribuer l'eau équitablement dans un quartier quand la demande est incertaine

Plan de projet et feuille de route du groupe

THÈME	Thème 3 · Réseaux, incertitude et optimisation
GROUPE	Groupe 1 · 6 membres (M1 à M6)
SUPERVISEUR	Ir. Charbel Mamlankou
STATUT	Document interne de cadrage, à usage du groupe
SOURCE	Basé sur le document officiel des sujets AMA

Table des matières

1. Vue d'ensemble du projet	3
2. Pipeline complet du projet	5
3. Notions à rechercher	10
4. Démonstrations mathématiques nécessaires	13
5. Données nécessaires	14
6. Expériences à réaliser	15
7. Architecture du projet proposée	18
8. Répartition détaillée entre les 6 membres	20
9. Matrice des responsabilités	23
10. Comprendre le projet	25
10.1 Ce que le projet demande, concrètement	25
10.2 Le rôle de chaque membre, en clair	25
10.3 Boîte à outils : quoi utiliser, et pourquoi	26
10.4 Qui attend qui : les dépendances, en clair	27
10.5 Livrables : à quoi reconnaît-on que c'est fini	27
11. Ordre chronologique du travail	29
12. Dépendances et bloqueurs (référence rapide)	30
13. Checklist finale	31
14. Risques et erreurs à éviter	32
15. Plan de travail pour les premiers jours	33
16. Prérequis avant de commencer l'implémentation	34

1 Vue d'ensemble du projet

Ce que le sujet demande réellement, avant d'entrer dans le détail des tâches.

1.1 Le problème et l'objectif

Le problème concret. Une société de distribution d'eau alimente plusieurs quartiers depuis quelques réservoirs, via un réseau de conduites à débit limité. Aujourd'hui, la répartition est faite de façon empirique, proportionnelle à la demande historique, sans réelle optimisation. La demande de chaque quartier varie en outre de façon imprévisible d'un jour à l'autre.

L'objectif final. Produire un outil qui calcule une répartition de l'eau plus équitable et moins coûteuse que la pratique actuelle, tout en restant fiable lorsque la demande réelle s'écarte des prévisions.

1.2 Entrées et sorties attendues

Entrées nécessaires

- Topologie du réseau : réservoirs, quartiers, conduites (qui relie quoi).
- Capacités des conduites (bornes sur les débits).
- Coût unitaire c_e de chaque conduite.
- Paramètres de la loi de demande de chaque quartier : μ_i , σ_i .
- Offre totale disponible aux réservoirs.

Sorties attendues

- Un vecteur de débits optimal q^* sur chaque conduite.
- Une comparaison chiffrée entre la distribution actuelle et q^* , sur plusieurs scénarios simulés.
- Une caractérisation de la robustesse de q^* face à l'incertitude (variance des résultats, quartiers à risque).

1.3 Hypothèses de modélisation

Imposées par le sujet. La demande est une variable aléatoire gaussienne, $D_i \sim N(\mu_i, \sigma_i^2)$. La contrainte d'égalité $Aq=b$ doit être traitée par pénalisation, jamais par multiplicateurs de Lagrange. La contrainte $q \geq 0$ est traitée par projection.

À définir par le groupe. Nombre de quartiers et de réservoirs, topologie précise du graphe, valeurs de c_e , μ_i et σ_i , niveau de corrélation entre quartiers voisins, nombre de tirages Monte-Carlo, pas d'apprentissage et paramètre de pénalisation μ , critère d'arrêt.

1.4 Contraintes mathématiques et formulation

- **Conservation des flux :** $Aq = b$, où A est la matrice d'incidence du graphe et q le vecteur des débits.
- **Positivité des débits :** $q \geq 0$.
- Le problème s'écrit $q^* = \operatorname{argmin}_q \sum c_e q_e^2$ sous ces deux contraintes.

1.5 Méthodes explicitement demandées par le sujet

- Représentation en graphe $G=(V, E)$ avec matrice d'incidence.
- Étude du rang et du conditionnement de A .

- Modélisation probabiliste de la demande par variable gaussienne.
- Simulation Monte-Carlo des scénarios de demande.
- Formulation d'optimisation quadratique sous contraintes.
- Pénalisation pour la contrainte d'égalité (jamais Lagrange : c'est explicitement interdit par le sujet).
- Projection pour la contrainte de positivité.
- Descente de gradient, éventuellement projetée.

1.6 Exigences imposées pour le rapport et pour le code

Six éléments doivent absolument apparaître dans le rapport, quelle que soit la façon dont le groupe organise son travail :

1. Une proposition mathématique justifiée (typiquement : convexité ou non de J).
2. Une dérivation complète de $\nabla J(q)$.
3. Un algorithme (descente de gradient ou variante) correctement justifié.
4. Une validation numérique confrontant explicitement théorie et expérience.
5. Une analyse de sensibilité d'au moins un hyperparamètre (candidat naturel : μ , ou le pas d'apprentissage).
6. Une comparaison à une stratégie de référence : la distribution actuelle, proportionnelle à la demande.

Règle stricte. Le rapport ne doit contenir aucun extrait de code source, uniquement du pseudo-code. Toute formule codée (gradient, seuil, critère d'arrêt...) doit avoir été dérivée à la main dans le rapport avant d'être codée.

À implémenter dans le code, livré séparément du rapport : construction du graphe et de A , génération de scénarios de demande, simulation Monte-Carlo, calcul de $J(q)$ et $\nabla J(q)$, descente de gradient projetée, calcul de la distribution actuelle de référence, comparaison des deux stratégies, production des figures et tableaux.

1.7 Expériences obligatoires et comparaison de référence

Au minimum deux résultats théoriques établis dans le rapport doivent être vérifiés empiriquement, par exemple la convergence de la descente de gradient sous la condition sur le pas, et l'effet du conditionnement de A sur la stabilité numérique. À cela s'ajoutent l'analyse de sensibilité d'un hyperparamètre et la comparaison à la stratégie de référence sur plusieurs scénarios.

Le livrable central attendu par le sujet est une comparaison chiffrée entre la distribution actuelle (proportionnelle aux demandes) et la distribution optimisée q^* , testée sur plusieurs scénarios de demande simulés. C'est le résultat final que l'outil doit produire.

Note sur la loi normale. Le sujet ne demande pas de justifier le choix de la loi normale par le théorème central limite, contrairement au Thème 1. Une justification courte reste toutefois attendue, car c'est une hypothèse de modélisation que le groupe doit défendre. Ce n'est pas une exigence stricte du texte, mais c'est fortement recommandé pour la rubrique « Probabilités et statistiques » (20 % de la note).

2 Pipeline complet du projet

Pipeline adapté à partir du pipeline méthodologique général du document AMA et des spécificités du Thème 3. Chaque étape précise pourquoi elle existe, ce qu'il faut comprendre et produire, et de quoi elle dépend.

GROUPE A : FONDATIONS (ÉTAPES 1 À 5)

Étape 1 Données et hypothèses réseau

Pourquoi : tout le reste du projet dépend d'une topologie de réseau figée.

Théorie	ce qu'est un graphe orienté, notion de nœud source/puits, notion de capacité.
À rechercher	représentation de réseaux de distribution (eau, électricité) comme graphes, conventions d'orientation des arêtes.
À calculer / démontrer	rien à ce stade : c'est une étape de définition.
À implémenter	structure de données du réseau (liste de nœuds, liste d'arêtes avec capacité et coût).

Dépendances : aucune, c'est le point de départ. | **Livrable** : schéma du réseau + fichier de données décrivant **V**, **E**, capacités, coûts.

Étape 2 Représentation en graphe $G=(V, E)$

Pourquoi : le sujet exige explicitement que le graphe soit un véritable élément mathématique du projet, pas une visualisation décorative.

Théorie	graphes orientés, connexité, composantes connexes, degré d'un nœud.
À rechercher	définitions formelles de connexité forte/faible, notion de point de fragilité (arête pont, nœud de faible degré).
À calculer / démontrer	vérifier la connexité du graphe construit, identifier les quartiers reliés par une seule conduite.
À implémenter	construction du graphe avec une bibliothèque adaptée, algorithme de test de connexité.

Dépendances : Étape 1. | **Livrable** : rapport de connexité, liste des points de fragilité.

Étape 3

Analyse algébrique (matrice d'incidence)

Pourquoi : le système $Aq=b$ est le cœur mathématique reliant graphe et optimisation.

Théorie construction de la matrice d'incidence orientée, interprétation de $Aq=b$ comme loi de conservation des flux à chaque nœud.

À rechercher matrice d'incidence, rang d'une matrice, noyau, conditionnement, valeurs propres de $A^T A$.

À calculer / démontrer rang de A en fonction de la structure du graphe, lien entre un réseau mal maillé et un mauvais conditionnement, donc une instabilité numérique.

À implémenter construction de A à partir du graphe, calcul du rang et du conditionnement.

Dépendances : Étape 2. | **Livable :** preuve écrite du lien structure/conditionnement, matrice A codée et testée.

Étape 4

Modélisation probabiliste de la demande

Pourquoi : la demande réelle n'est jamais connue à l'avance : c'est la difficulté centrale du sujet.

Théorie variables aléatoires gaussiennes, espérance, variance, corrélation entre variables.

À rechercher loi normale, justification qualitative (somme de nombreuses petites causes indépendantes), estimateurs de μ et σ , corrélation empirique.

À calculer / démontrer formules des estimateurs (moyenne empirique, variance empirique), intervalle de confiance sur μ_i .

À implémenter génération de tirages $D_i \sim N(\mu_i, \sigma_i^2)$, calcul des corrélations entre quartiers voisins.

Dépendances : Étape 1 (nombre de quartiers et demandes typiques). | **Livable :** modèle de demande calibré, section rapport correspondante.

Étape 5

Simulation Monte-Carlo

Pourquoi : transformer le modèle probabiliste en scénarios exploitables pour tester la robustesse de la solution.

Théorie loi des grands nombres, convergence des estimateurs empiriques avec le nombre de tirages, notion d'erreur d'estimation Monte-Carlo.

À rechercher méthode Monte-Carlo, choix du nombre de tirages, calcul d'intervalles de confiance sur des statistiques simulées.

À calculer / démontrer ordre de grandeur de l'erreur d'estimation en fonction du nombre de tirages, en $1/\sqrt{N}$.

À implémenter génération de N scénarios de demande, calcul de statistiques agrégées (moyenne, variance, quantiles).

Dépendances : Étape 4. | **Livable :** jeu de scénarios de demande, statistiques descriptives associées.

GROUPE B : FORMULATION ET ALGORITHMES (ÉTAPES 6 À 9)

Étape 6

Formulation de l'optimisation

Pourquoi : c'est l'objectif final mathématique, $q^* = \operatorname{argmin}_q \sum c_e q_e^2$ sous $Aq=b$, $q \geq 0$.

Théorie optimisation quadratique sous contraintes linéaires, notion de fonction convexe.

À rechercher conditions de convexité d'une forme quadratique, notion de problème bien posé.

À calculer / démontrer preuve que $\sum c_e q_e^2$ est convexe, car $c_e > 0$.

À implémenter rien à ce stade : c'est une étape de formulation mathématique.

Dépendances : Étape 3 (il faut A et b). | **Livrable :** formulation mathématique complète et validée par le groupe.

Étape 7

Pénalisation

Pourquoi : la contrainte méthodologique impose de traiter $Aq=b$ par pénalisation plutôt que par Lagrange, pour rester dans le cadre de la descente de gradient.

Théorie méthode de pénalisation quadratique, comportement de la solution pénalisée quand $\mu \rightarrow \infty$.

À rechercher pénalisation en optimisation, compromis entre respect de la contrainte et conditionnement du problème pénalisé.

À calculer / démontrer $J(q) = \sum c_e q_e^2 + \mu \|Aq-b\|^2$, preuve de convexité de J , comportement asymptotique en μ .

À implémenter fonction de coût pénalisée, en code.

Dépendances : Étapes 3 et 6. | **Livrable :** dérivation complète et justifiée de $J(q)$ dans le rapport.

Étape 8

Gradient

Pourquoi : la contrainte méthodologique exige une dérivation complète de ∇J .

Théorie dérivation de fonctions quadratiques vectorielles, règles de dérivation matricielle.

À rechercher gradient d'une forme quadratique $x^T C x$, gradient de $\|Ax-b\|^2$.

À calculer / démontrer $\nabla J(q) = 2Cq + 2\mu A^T(Aq-b)$, où $C = \operatorname{diag}(c_e)$, dérivation ligne par ligne à présenter dans le rapport.

À implémenter fonction de calcul du gradient.

Dépendances : Étape 7. | **Livrable :** dérivation manuelle complète, vérifiée par au moins deux membres.

Étape 9 Descente de gradient et projection

Pourquoi : c'est l'algorithme demandé pour résoudre le problème pénalisé sous la contrainte $q \geq 0$.

Théorie règle de mise à jour $q_{k+1} = q_k - \eta \nabla J(q_k)$, projection euclidienne sur l'orthant positif ($\max(\cdot, 0)$ composante par composante), conditions de convergence liées à la constante de Lipschitz du gradient.

À rechercher descente de gradient projetée, choix du pas d'apprentissage, critère d'arrêt, constante de Lipschitz d'une forme quadratique.

À calculer / démontrer borne théorique sur le pas $\eta < 2/L$ avec L la plus grande valeur propre de $2C + 2\mu A^T A$, justification que $\max(\cdot, 0)$ est bien la projection euclidienne sur $\{q \geq 0\}$.

À implémenter boucle de descente de gradient avec projection à chaque itération, suivi de la norme du gradient et de la violation de contrainte.

Dépendances : Étape 8. | **Livrable** : algorithme validé, pseudo-code dans le rapport, code fonctionnel séparé.

GROUPE C : EXPÉRIMENTATION, VALIDATION ET LIVRAISON (ÉTAPES 10 À 14)

Étape 10 Génération de scénarios de test

Pourquoi : il faut tester q^* sur des situations variées, pas uniquement sur la demande moyenne.

Théorie reprise des notions Monte-Carlo, notion de scénario de stress (demande extrême).

À rechercher construction de scénarios représentatifs (moyen, favorable, défavorable).

À calculer / démontrer rien de nouveau : réutilisation de l'Étape 5.

À implémenter script qui exécute la résolution sur chaque scénario généré.

Dépendances : Étapes 5 et 9. | **Livrable** : ensemble de résultats q^* par scénario.

Étape 11 Comparaison à la stratégie de référence

Pourquoi : c'est le livrable explicite du sujet : comparaison chiffrée entre distribution actuelle et q^* .

Théorie définition précise de la stratégie actuelle (répartition proportionnelle à la demande), notion de métrique de comparaison (coût total, équité entre quartiers, violation de contrainte).

À rechercher indicateurs d'équité (par exemple écart-type de satisfaction par quartier), notion de coût social contre coût technique.

À calculer / démontrer formule de la stratégie de référence, définition précise des métriques utilisées.

À implémenter calcul de la stratégie proportionnelle, calcul des métriques de comparaison sur tous les scénarios.

Dépendances : Étape 10. | **Livrable** : tableau et graphiques comparatifs.

Étape 12

Validation théorie / expérience

Pourquoi : la contrainte méthodologique exige explicitement une confrontation entre théorie et expérience.

Théorie	reprise des résultats théoriques de convergence et de conditionnement établis plus haut.
À rechercher	méthodes de vérification empirique d'un résultat théorique (courbe de convergence, mesure d'erreur résiduelle).
À calculer / démontrer	vérifier numériquement que la convergence respecte la borne théorique sur le pas, vérifier que le conditionnement mesuré correspond à l'analyse algébrique de l'Étape 3.
À implémenter	scripts d'expérience dédiés, voir section 6.

Dépendances : Étapes 9 et 3. | **Livrable :** au moins deux confrontations théorie/expérience documentées.

Étape 13

Analyse des résultats

Pourquoi : le sujet demande une interprétation mathématique, pas seulement descriptive.

Théorie	lecture critique des résultats à la lumière des hypothèses posées.
À rechercher	rien de nouveau : synthèse.
À calculer / démontrer	discussion de l'effet de μ , discussion des limites du modèle gaussien pour la demande.
À implémenter	rien : c'est une étape de rédaction.

Dépendances : Étapes 11 et 12. | **Livrable :** section « Analyse des résultats » du rapport.

Étape 14

Outil final

Pourquoi : c'est le livrable technique demandé en plus du rapport.

Théorie	aucune nouvelle notion : assemblage des briques précédentes.
À rechercher	bonnes pratiques de structuration d'un petit projet Python.
À calculer / démontrer	rien.
À implémenter	script ou notebook exécutable de bout en bout, prenant un réseau et des paramètres de demande en entrée et produisant la comparaison finale.

Dépendances : toutes les étapes précédentes. | **Livrable :** outil fonctionnel + README.

3 Notions à rechercher

Liste des notions à maîtriser avant de pouvoir dériver et coder correctement chaque brique du projet, classées par thème et par priorité.

Théorie des graphes

Notion	Priorité
Graphe orienté, nœuds, arêtes	Indispensable
Matrice d'incidence d'un graphe orienté	Indispensable
Connexité, composantes connexes	Indispensable
Points de fragilité (arêtes pont, nœuds de faible degré)	Important
Chemins, cycles dans un graphe orienté	Secondaire

Algèbre linéaire

Notion	Priorité
Construction de A à partir du graphe	Indispensable
Interprétation de $Aq=b$ comme conservation des flux	Indispensable
Rang d'une matrice, lien avec la connexité du graphe	Indispensable
Noyau de A	Important
Conditionnement $\kappa(A)$ ou $\kappa(A^T A)$	Indispensable
Valeurs propres d'une matrice symétrique définie positive	Important
Lien entre structure du réseau (maillage) et stabilité numérique	Indispensable

Probabilités

Notion	Priorité
Variable aléatoire gaussienne, densité, espérance, variance	Indispensable
Justification qualitative du choix de la loi normale	Important
Corrélation entre variables aléatoires, covariance	Important
Somme de variables gaussiennes corrélées	Secondaire

Statistiques

Notion	Priorité
Estimateur de la moyenne et de la variance	Indispensable
Intervalle de confiance sur une moyenne	Indispensable
Estimation de corrélations empiriques	Important
Détection de quartiers à demande atypique	Important

Monte-Carlo

Notion	Priorité
Principe de la simulation Monte-Carlo	Indispensable
Loi des grands nombres, convergence des estimateurs empiriques	Indispensable
Choix du nombre de tirages, erreur d'estimation en $1/\sqrt{N}$	Important
Réduction de variance (échantillonnage stratifié, etc.)	Secondaire

Optimisation

Notion	Priorité
Optimisation quadratique sous contraintes linéaires	Indispensable
Convexité d'une fonction quadratique	Indispensable
Méthode de pénalisation, et pourquoi pas Lagrange ici	Indispensable
Comportement de la solution pénalisée quand μ varie	Indispensable

Descente de gradient

Notion	Priorité
Règle de mise à jour de la descente de gradient	Indispensable
Choix du pas d'apprentissage	Indispensable
Constante de Lipschitz du gradient, lien avec le pas maximal admissible	Important
Variantes (momentum, pas adaptatif)	Secondaire

Pénalisation

Notion	Priorité
Construction du terme de pénalisation quadratique	Indispensable
Effet de μ sur la précision du respect de la contrainte	Indispensable

Notion	Priorité
Effet de μ sur le conditionnement du problème pénalisé	Important

Projection

Notion	Priorité
Projection euclidienne sur l'orthant positif	Indispensable
Justification que $\max(\cdot, 0)$ réalise cette projection	Indispensable
Descente de gradient projetée, convergence	Important

Convergence

Notion	Priorité
Conditions suffisantes de convergence sur fonction convexe et lisse	Indispensable
Critère d'arrêt (norme du gradient, variation de J , itérations max)	Indispensable
Vitesse de convergence en fonction du conditionnement	Important

Validation numérique

Notion	Priorité
Méthodologie de confrontation théorie/expérience	Indispensable
Construction de graphiques de convergence	Indispensable
Analyse de sensibilité d'un hyperparamètre	Indispensable

4 Démonstrations mathématiques nécessaires

Ces démonstrations doivent être prêtes **avant** que quiconque commence à coder les fonctions correspondantes : le code ne précède jamais la théorie.

Démonstration	Responsable suggéré	Notions à maîtriser avant de commencer
Interprétation physique de $Aq=b$ comme bilan des flux à chaque nœud	Membre 1	Graphe orienté, matrice d'incidence
Construction formelle de A , lien lignes/colonnes avec nœuds/arêtes	Membre 2	Matrice d'incidence
Rang de A en fonction de la connexité du graphe	Membre 2	Rang d'une matrice, connexité
Conditionnement de A (ou $A^T A$), lien avec le maillage du réseau	Membre 2	Valeurs propres, conditionnement
Justification du choix de la loi normale pour D_i	Membre 3	Variable aléatoire gaussienne
Estimateurs de μ_i et σ_i , intervalle de confiance	Membre 3	Statistiques d'estimation
Convexité de $J(q) = \sum c_e q_e^2 + \mu \ Aq-b\ ^2$	Membre 4	Fonctions quadratiques convexes
Dérivation complète de $\nabla J(q)$	Membre 4 (revu par M2)	Dérivation matricielle
Règle de mise à jour de la descente de gradient	Membre 4	Optimisation itérative
Projection sur $q \geq 0$ (justifier $\max(\cdot, 0)$)	Membre 4	Projection euclidienne
Conditions de convergence (pas maximal, critère d'arrêt)	Membre 4 (revu par M5)	Constante de Lipschitz
Influence de μ (respect de la contrainte contre conditionnement)	Membre 4	Pénalisation
Lien entre résultats théoriques et résultats numériques	Membre 6 (avec M4)	Synthèse de tout ce qui précède

Chaque démonstration doit être relue par au moins une deuxième personne avant d'être considérée comme validée pour le rapport.

5 Données nécessaires

5.1 Structure minimale de données

- Nombre de quartiers $|V_{\text{quartiers}}|$ et nombre de réservoirs $|V_{\text{réservoirs}}|$.
- Liste des conduites (arêtes), chacune avec : nœud de départ, nœud d'arrivée, capacité maximale, coût unitaire c_e .
- Pour chaque quartier : demande moyenne μ_i , variance σ_i^2 .
- Éventuellement : matrice de corrélation entre quartiers voisins.
- Éventuellement : historique de demande observée, pour comparer aux prévisions et calibrer μ_i , σ_i .
- Distribution actuelle : règle explicite (par exemple proportionnelle à la demande moyenne) servant de référence.

5.2 Données réelles ou synthétiques

Le sujet ne demande pas de données réelles. Un jeu de données synthétique réaliste est justifiable et recommandé, à condition que sa construction soit expliquée et défendue dans le rapport.

Construction recommandée

- Générer un graphe connexe avec un nombre raisonnable de quartiers (8 à 15) et de réservoirs (2 à 3).
- Fixer des capacités et des coûts cohérents avec la taille du réseau.
- Choisir des μ_i et σ_i plausibles, avec un coefficient de variation raisonnable, par exemple σ_i entre 10 % et 30 % de μ_i .
- Introduire une corrélation modérée entre quartiers géographiquement proches.

Il n'est pas nécessaire de chercher des données réelles de réseau d'eau : cela ajouterait de la complexité sans être exigé par le sujet.

6 Expériences à réaliser

Six expériences numériques couvrent l'ensemble des exigences du sujet : comparaison de référence, robustesse, sensibilité, et double confrontation théorie/expérience.

EXPÉRIENCE 1

Distribution actuelle contre q^* , sur la demande moyenne

Objectif : mesurer le gain de la solution optimisée dans le cas le plus simple.

Paramètres	$D_i = \mu_i$ (pas d'aléa), μ de pénalisation fixé à une valeur de référence.
Données	réseau synthétique complet.
Méthode	résolution par descente de gradient projetée, calcul de la distribution proportionnelle de référence.
Métriques	coût total $\sum c_e q_e^2$, violation résiduelle $\ Aq - b\ $, écart-type de satisfaction entre quartiers.
Résultat attendu	q^* a un coût inférieur ou égal à la référence, avec une violation de contrainte proche de zéro.
Graphique	diagramme en barres comparant les deux stratégies par quartier.
Interprétation	discuter pourquoi la solution proportionnelle n'est pas optimale au sens de $\sum c_e q_e^2$.

EXPÉRIENCE 2

Robustesse sur plusieurs scénarios Monte-Carlo

Objectif : évaluer la stabilité des deux stratégies face à l'incertitude.

Paramètres	N tirages (par exemple 1000) de $D_i \sim N(\mu_i, \sigma_i^2)$.
Données	mêmes réseau et lois que l'Expérience 1.
Méthode	résoudre le problème pour chaque scénario, agréger les résultats.
Métriques	moyenne et variance du coût total sur les scénarios, taux de scénarios où la contrainte est bien respectée.
Résultat attendu	q^* reste globalement meilleur ou plus stable que la référence sur l'ensemble des scénarios.
Graphique	histogramme de la distribution des coûts pour les deux stratégies.
Interprétation	lien avec la loi des grands nombres et la fiabilité statistique de la comparaison.

EXPÉRIENCE 3

Analyse de sensibilité de μ (hyperparamètre de pénalisation)

Objectif : satisfaire l'exigence explicite d'analyse de sensibilité d'un hyperparamètre.

Paramètres	plusieurs valeurs de μ (par exemple 1, 10, 100, 1000).
Données	réseau synthétique, demande moyenne.
Méthode	résoudre le problème pénalisé pour chaque μ , observer convergence et violation résiduelle.
Métriques	violation résiduelle finale $\ Aq - b\ $, nombre d'itérations avant convergence, conditionnement effectif.
Résultat attendu	violation résiduelle qui décroît avec μ , mais convergence qui ralentit pour μ trop grand.
Graphique	courbes de convergence superposées pour chaque μ .
Interprétation	compromis à discuter explicitement, en lien avec la dérivation théorique de l'Étape 7.

EXPÉRIENCE 4

Vérification empirique de la convergence théorique

Objectif : confronter la borne théorique sur le pas d'apprentissage à un comportement observé.

Paramètres	plusieurs pas η , certains respectant la borne théorique $\eta < 2/L$, d'autres non.
Données	réseau synthétique.
Méthode	lancer la descente de gradient projetée pour chaque η , suivre la norme du gradient au fil des itérations.
Métriques	norme du gradient, valeur de $J(q)$ au fil des itérations.
Résultat attendu	convergence stable sous la borne théorique, oscillation ou divergence au-delà.
Graphique	courbes de $J(q_k)$ en fonction de k , pour différents η .
Interprétation	validation directe du résultat théorique de l'Étape 9.

EXPÉRIENCE 5

Effet du maillage du réseau sur le conditionnement

Objectif : vérifier empiriquement le lien entre structure du graphe et stabilité numérique établi à l'Étape 3.

Paramètres	plusieurs variantes du réseau (peu de conduites, réseau bien maillé).
Données	réseaux synthétiques générés avec des densités de connexion différentes.
Méthode	calculer $\kappa(A)$ pour chaque variante, résoudre le problème et observer le nombre d'itérations nécessaires.
Métriques	conditionnement, nombre d'itérations à convergence.
Résultat attendu	un réseau mal maillé a un conditionnement plus élevé et nécessite davantage d'itérations.
Graphique	conditionnement en fonction du nombre de conduites, ou itérations en fonction du conditionnement.
Interprétation	deuxième confrontation théorie/expérience, complète l'Expérience 4.

EXPÉRIENCE 6

Identification des quartiers à risque

Objectif : exploiter le volet statistique pour enrichir l'interprétation.

Paramètres	historique simulé de demande sur plusieurs périodes.
Données	tirages répétés avec un biais volontaire sur certains quartiers.
Méthode	calcul d'intervalles de confiance et de corrélations, comparaison à la demande prévue.
Métriques	fréquence de dépassement de l'intervalle de confiance par quartier.
Résultat attendu	identification de quartiers dont la demande s'écarte régulièrement des prévisions.
Graphique	tableau récapitulatif par quartier.
Interprétation	lien avec la robustesse de q^* pour ces quartiers spécifiques.

7 Architecture du projet proposée

Une seule arborescence de dépôt de code pour tout le groupe, pour éviter que chacun ne réinvente sa propre structure.

```
project/
├─ data/
│   ├── generate_network.py      # génère un réseau synthétique réaliste
│   └─ network_config.json      # V, E, capacités, coûts, mu_i, sigma_i
├─ notebooks/
│   └─ exploration.ipynb        # exploration libre, non livrée comme résultat final
├─ src/
│   ├── graph/
│   │   ├── build_graph.py      # construit G et la matrice d'incidence A
│   │   └─ graph_analysis.py    # connexité, points de fragilité, rang, conditionnement
│   ├── probability/
│   │   ├── demand_model.py     # définition et calibration de  $D_i \sim N(\mu_i, \sigma_i^2)$ 
│   │   └─ monte_carlo.py       # génération de scénarios, statistiques agrégées
│   ├── optimization/
│   │   ├── objective.py        #  $J(q)$  et  $\text{gradient}(q)$ 
│   │   └─ gradient_descent.py  # descente de gradient projetée
│   ├── simulation/
│   │   └─ run_scenarios.py      # orchestration : résolution sur chaque scénario
│   └─ evaluation/
│       ├── baseline.py         # distribution actuelle (proportionnelle)
│       ├── metrics.py          # coût, équité, violation de contrainte
│       └─ compare_strategies.py
├─ experiments/
│   ├── exp1_baseline_vs_optimal.py
│   ├── exp2_monte_carlo_robustness.py
│   ├── exp3_sensitivity_mu.py
│   ├── exp4_convergence_check.py
│   ├── exp5_conditioning_vs_mesh.py
│   └─ exp6_at_risk_districts.py
├─ results/
│   ├── figures/
│   └─ tables/
├─ report/
│   └─ (rapport final, sans code source)
└─ README.md
```

Responsabilités des modules principaux

Module	Rôle
graph/build_graph.py	Construit G à partir de <code>network_config.json</code> , retourne A .
graph/graph_analysis.py	Tests de connexité, détection des points de fragilité, calcul du rang et du conditionnement.
probability/demand_model.py	Stocke μ_i , σ_i , fonctions d'échantillonnage et d'estimation.
probability/monte_carlo.py	Génère N scénarios, calcule statistiques agrégées et intervalles de confiance.

Module	Rôle
<code>optimization/objective.py</code>	Implémente $J(q)$ et $\nabla J(q)$ conformément à la dérivation validée dans le rapport.
<code>optimization/gradient_descent.py</code>	Boucle d'optimisation avec projection, suivi de convergence, critère d'arrêt.
<code>evaluation/baseline.py</code>	Calcule la distribution proportionnelle de référence.
<code>evaluation/metrics.py</code> , <code>compare_strategies.py</code>	Centralisent toutes les métriques de comparaison utilisées dans les expériences.

8 Répartition détaillée entre les 6 membres

La répartition ci-dessous est construite à partir de la charge réelle de travail, en s'appuyant sur les poids de la grille d'évaluation officielle. Elle s'écarte volontairement de la répartition « type » suggérée en introduction du sujet, pour éviter que les rubriques les plus lourdes (probabilités et optimisation, 40 % à elles deux) ne reposent sur une seule personne.

Rubrique de la grille d'évaluation	Poids
Formulation	10 %
Algèbre linéaire	15 %
Probabilités et statistiques	20 %
Optimisation	20 %
Mathématiques discrètes	10 %
Implémentation et validation	15 %
Rédaction et rigueur	10 %

Membre 1 Modélisation du problème et réseau

MISSION : fixer la topologie du réseau (quartiers, réservoirs, conduites), les hypothèses physiques, l'orientation des arêtes, les capacités. Construire et défendre l'interprétation physique de $Aq=b$. Vérifier la connexité du graphe avec Membre 2.

LIVRABLES VÉRIFIABLES : fichier de configuration du réseau, schéma du graphe, section « Formulation du problème » du rapport (variables, hypothèses, contraintes).

DÉPENDANCES : bloque Membre 2 (qui a besoin du graphe pour construire A) et, indirectement, tout le reste du projet.

Membre 2 Algèbre linéaire et structure discrète

MISSION : construire formellement A à partir du graphe de Membre 1, étudier son rang, son conditionnement, démontrer le lien entre maillage du réseau et stabilité numérique. Contribuer à la détection des points de fragilité.

LIVRABLES VÉRIFIABLES : démonstrations écrites (rang, conditionnement), module `graph_analysis.py`, section « Représentation algébrique » et contribution à « Structure discrète » du rapport.

DÉPENDANCES : dépend de Membre 1. Alimente Membre 4 (le conditionnement influence le choix du pas) et Membre 6 (Expérience 5).

Membre 3 Probabilités et statistiques

MISSION : justifier et calibrer le modèle $D_i \sim N(\mu_i, \sigma_i^2)$, définir les estimateurs de μ_i , σ_i , construire les intervalles de confiance, étudier les corrélations entre quartiers voisins, identifier les quartiers à demande atypique.

LIVRABLES VÉRIFIABLES : module `demand_model.py`, tableau des estimateurs et intervalles de confiance, section « Modélisation probabiliste » et « Analyse statistique » du rapport.

DÉPENDANCES : dépend de Membre 1 (nombre de quartiers, ordres de grandeur). Alimente Membre 5 (paramètres nécessaires au Monte-Carlo).

Membre 4 Optimisation et dérivation du gradient

MISSION : formuler $J(q)$ pénalisé, démontrer sa convexité, dériver $\nabla J(q)$ complètement à la main, écrire la règle de mise à jour et la projection, établir les conditions de convergence (borne sur le pas, critère d'arrêt), analyser l'influence de μ .

LIVRABLES VÉRIFIABLES : dérivations manuscrites complètes et relues, pseudo-code de l'algorithme, section « Formulation d'optimisation » du rapport.

DÉPENDANCES : dépend de Membre 2 (a besoin de A et de son conditionnement pour la borne sur le pas). Bloque Membre 5 (implémentation du solveur).

Membre 5 Monte-Carlo, implémentation et expérimentation

MISSION : implémenter `monte_carlo.py`, coder `objective.py` et `gradient_descent.py` à partir des dérivations de Membre 4, exécuter toutes les expériences, produire les figures et tableaux.

LIVRABLES VÉRIFIABLES : code fonctionnel testé, résultats bruts de toutes les expériences, figures dans `results/figures`.

DÉPENDANCES : dépend de Membre 3 (modèle de demande) et de Membre 4 (dérivations validées). Alimente Membre 6.

Membre 6 Validation, comparaison, intégration et rédaction

MISSION : construire `compare_strategies.py` et `baseline.py`, confronter les résultats numériques de Membre 5 aux prédictions théoriques de Membre 4 et Membre 2, rédiger le rapport complet (introduction, méthode numérique, analyse des résultats, limites, conclusion, bibliographie), vérifier la conformité aux contraintes méthodologiques AMA, préparer la checklist finale et la présentation.

LIVRABLES VÉRIFIABLES : rapport complet assemblé, checklist de conformité remplie, support de présentation.

DÉPENDANCES : dépend de tous les autres membres : c'est la dernière étape avant la remise.

Équilibre de la charge. Membre 4 porte la partie la plus dense théoriquement (optimisation, 20 %) mais avec un périmètre de code réduit, délégué à Membre 5. Membre 3 porte l'autre rubrique à 20 % avec un périmètre de code également réduit. Membre 5 porte une charge d'implémentation lourde mais peu de dérivation théorique. Membre 6 porte la rédaction (10 %) mais aussi une part réelle de validation numérique (15 % partagée avec Membre 5), ce qui évite qu'il ne fasse « que » de la rédaction.

9 Matrice des responsabilités

Vue synthétique de toutes les tâches du projet : qui fait quoi, avec quelle dépendance, pour quel livrable, et dans quelle partie du rendu (théorie, code, rapport) la tâche apparaît.

Tâche	Priorité	Resp.	Collab.	Dépendance	Livrable	Théorie	Code	Rapport
Définir topologie du réseau	Indispensable	M1	M2	Aucune	Config réseau	✓	–	✓
Construire le graphe et vérifier la connexité	Indispensable	M1	M2	Topologie	Graphe validé	✓	✓	✓
Construire A , étudier rang et conditionnement	Indispensable	M2	M1	Graphe	Preuve écrite + module	✓	✓	✓
Modéliser $D_i \sim N(\mu_i, \sigma_i^2)$ et le justifier	Indispensable	M3	M6	Topologie	Modèle calibré	✓	–	✓
Estimateurs de μ_i , σ_i , IC	Indispensable	M3	–	Modèle demande	Tableau estimateurs	✓	✓	✓
Étudier corrélations entre quartiers	Important	M3	–	Modèle demande	Matrice de corrélation	✓	✓	✓
Formuler $J(q)$ pénalisé et prouver sa convexité	Indispensable	M4	M2	Matrice A	Preuve écrite	✓	–	✓
Dériver $\nabla J(q)$	Indispensable	M4	M2	Formulation J	Dérivation manuscrite	✓	–	✓
Écrire règle de mise à jour et projection	Indispensable	M4	–	Gradient dérivé	Pseudo-code	✓	–	✓
Établir les conditions de convergence	Indispensable	M4	M2	Conditionnement de A	Borne théorique sur le pas	✓	–	✓
Implémenter le générateur Monte-Carlo	Indispensable	M5	M3	Modèle demande	<code>monte_carlo.py</code>	–	✓	–
Implémenter <code>objective.py</code> et <code>gradient_descent.py</code>	Indispensable	M5	M4	Dérivations validées	Solveur fonctionnel	–	✓	–

Tâche	Priorité	Resp.	Collab.	Dépendance	Livrable	Théorie	Code	Rapport
Implémenter la distribution de référence	Indispensable	M5	M6	Topologie, demande	baseline.py	–	✓	–
Exécuter les 6 expériences numériques	Indispensable	M5	M6	Solveur fonctionnel	Résultats bruts, figures	–	✓	–
Comparer stratégie actuelle et q^*	Indispensable	M6	M5	Résultats expériences	Tableau comparatif	✓	✓	✓
Confronter théorie et expérience (2 résultats min.)	Indispensable	M6	M2, M4	Résultats expériences	Section validation	✓	–	✓
Analyse de sensibilité de μ	Indispensable	M6	M4, M5	Solveur fonctionnel	Courbes, discussion	✓	✓	✓
Rédaction complète du rapport	Indispensable	M6	Tous	Toutes sections précédentes	Rapport final	–	–	✓
Vérification conformité contraintes AMA	Indispensable	M6	Tous	Rapport rédigé	Checklist remplie	–	–	✓
Préparation présentation / démonstration	Important	M6	Tous	Outil final	Support de présentation	–	–	–

10 Comprendre le projet

Section pédagogique. Elle reprend tout ce qui précède avec des mots simples : ce que le projet demande vraiment, ce que chacun doit produire, avec quels outils, et comment savoir qu'une tâche est vraiment terminée.

10.1 Ce que le projet demande, concrètement

Une société d'eau distribue de l'eau à plusieurs quartiers depuis quelques réservoirs, à travers des tuyaux qui ont chacun un débit maximal. Aujourd'hui, elle partage l'eau à peu près proportionnellement à ce que chaque quartier a l'habitude de consommer, sans vraiment calculer si ce partage est le meilleur possible. Le problème, c'est que personne ne connaît à l'avance la demande exacte de chaque quartier pour le lendemain : elle varie.

Le travail du groupe consiste à construire, à la main puis en code, une meilleure façon de répartir l'eau dans les tuyaux, une façon qui coûte moins cher et reste raisonnable même quand la demande réelle diffère de ce qui était prévu. Concrètement, le groupe doit produire deux choses :

1. Un **rapport écrit** qui pose le problème en mathématiques, démontre les résultats à la main (pas seulement de façon numérique), et confronte ces résultats à des expériences.
2. Un **outil informatique** (code Python) qui prend en entrée la description d'un réseau et d'une demande, et qui ressort une comparaison chiffrée entre l'ancienne méthode et la nouvelle.

La règle la plus importante à retenir : **on ne code jamais une formule avant de l'avoir dérivée à la main dans le rapport**. La théorie vient toujours en premier, le code vient ensuite pour l'appliquer.

10.2 Le rôle de chaque membre, en clair

M1 dessine le terrain de jeu. Il ou elle décide combien il y a de quartiers et de réservoirs, comment les tuyaux les relient, et quelles sont leurs limites. Sans ce choix, personne d'autre ne peut vraiment démarrer.

M2 vérifie que le terrain de jeu est solide. À partir du réseau de M1, il ou elle construit la matrice qui représente le réseau en mathématiques, et calcule si cette matrice est « bien élevée » (facile à travailler numériquement) ou non.

M3 s'occupe de l'incertitude. Il ou elle décide comment représenter le fait que la demande de chaque quartier n'est jamais connue avec certitude, et calibre des valeurs réalistes.

M4 fait les mathématiques du cœur du problème.

Il ou elle écrit, à la main, comment on doit corriger petit à petit une solution pour qu'elle devienne la meilleure possible, et prouve que la méthode fonctionne.

M5 fait tourner la machine. Il ou elle transforme en code ce que M3 et M4 ont écrit sur papier, et lance toutes les expériences qui produisent des chiffres et des graphiques.

M6 assemble tout et vérifie que rien ne manque. Il ou elle compare les résultats aux prévisions théoriques, rédige le rapport final, et s'assure que toutes les règles du sujet sont respectées.

10.3 Boîte à outils : quoi utiliser, et pourquoi

Le sujet impose une méthode (graphe, pénalisation, descente de gradient dérivée à la main). Ce tableau indique, pour chaque brique du projet, quel outil Python utiliser, à quoi il sert concrètement, et pourquoi il est adapté ici.

Tâche concernée	Outil ou méthode	À quoi ça sert	Pourquoi c'est le bon choix ici
Construction et analyse du graphe (M1, M2)	<code>networkx</code>	Représenter les nœuds et les arêtes, tester la connexité, repérer les points faibles du réseau.	Évite de recoder des algorithmes de graphes déjà connus et documente une structure claire pour tout le monde.
Matrice d'incidence, rang, conditionnement (M2)	<code>numpy</code> , <code>scipy.linalg</code>	Construire A , calculer son rang (<code>matrix_rank</code>) , son conditionnement (<code>cond</code>) et ses valeurs propres (<code>eigvalsh</code>) .	Ce sont des opérations d'algèbre linéaire standards, déjà optimisées et fiables ; les recoder serait une perte de temps et une source d'erreurs.
Modèle de demande (M3)	<code>numpy.random</code> , <code>scipy.stats</code>	Générer des tirages aléatoires D_i suivant une loi normale, calculer moyenne, variance, intervalles de confiance.	Le sujet impose explicitement une loi gaussienne ; ces bibliothèques génèrent et estiment de façon fiable.
Corrélations entre quartiers (M3)	<code>pandas</code> / <code>numpy.corrcoef</code>	Calculer une matrice de corrélation empirique entre demandes de quartiers voisins.	Nécessaire pour justifier, ou écarter, l'hypothèse de corrélation dans le modèle.
Simulation Monte-Carlo (M5, avec M3)	Boucle Python vectorisée avec <code>numpy</code>	Générer un grand nombre de scénarios de demande et calculer des statistiques agrégées.	Le sujet impose la méthode Monte-Carlo ; <code>numpy</code> le fait efficacement même avec des milliers de tirages.
Fonction de coût et gradient (M5, à partir de M4)	<code>numpy</code> pur, pas de solveur boîte noire	Coder exactement $J(q)$ et $\nabla J(q)$ tels que dérivés à la main.	Le sujet interdit un solveur boîte noire de type <code>scipy.optimize</code> : l'algorithme doit être écrit par le groupe, à partir de sa propre dérivation.
Descente de gradient projetée (M5)	Boucle Python, <code>max(., 0)</code> pour la projection	Faire converger q vers une solution qui respecte $q \geq 0$.	C'est l'algorithme explicitement demandé ; coder la projection soi-même montre la compréhension de la méthode.
Comparaison et métriques (M6, avec M5)	<code>pandas</code>	Organiser les résultats des différentes stratégies et scénarios dans des tableaux comparables.	Facilite le calcul des métriques et la production de tableaux propres pour le rapport.
Figures et graphiques (M5, M6)	<code>matplotlib</code>	Produire les courbes de convergence, histogrammes et diagrammes en barres des expériences.	Outil standard pour illustrer des résultats numériques dans un rapport scientifique.

Tâche concernée	Outil ou méthode	À quoi ça sert	Pourquoi c'est le bon choix ici
Vérification du code (tous, surtout M5)	<code>pytest</code>	Écrire de petits tests automatiques, par exemple : la projection ne renvoie jamais de valeur négative.	Donne une garantie concrète que chaque brique fonctionne avant de l'assembler dans le pipeline complet.
Exploration libre (M5)	Jupyter Notebook	Essayer rapidement des idées et visualiser des résultats intermédiaires.	Accélère le développement, mais ne remplace jamais le code propre et documenté du dossier <code>src/</code> .
Gestion du code partagé (tous)	Git + dépôt partagé (GitHub)	Garder une trace des modifications et permettre à plusieurs membres de travailler sur le même code.	Indispensable dès que plus d'une personne touche au même projet de code.
Rédaction du rapport (M6, avec tous)	LaTeX (recommandé), ou traitement de texte classique	Rédiger un document structuré avec des formules mathématiques lisibles.	Un rapport riche en formules est plus simple à mettre en forme et à corriger en LaTeX.

10.4 Qui attend qui : les dépendances, en clair

Le projet ne peut pas avancer dans n'importe quel ordre. Voici la chaîne logique à respecter :

- M1 fixe le réseau** (quartiers, réservoirs, conduites). Rien d'autre ne peut sérieusement démarrer avant cette étape.
- Une fois le réseau fixé, deux branches avancent en parallèle : **M2** construit la matrice `A` et calcule son rang et son conditionnement, pendant que **M3** calibre le modèle de demande.
- M4** peut commencer la dérivation théorique (convexité, gradient) dès que la formulation pénalisée est validée par le groupe, sans attendre M2, mais il ou elle a besoin du conditionnement calculé par M2 pour fixer une borne fiable sur le pas d'apprentissage.
- M5** ne peut coder le solveur d'optimisation qu'une fois les dérivations de M4 validées et relues : le code ne précède jamais la théorie. M5 ne peut lancer Monte-Carlo qu'une fois le modèle de M3 figé.
- La comparaison entre distribution actuelle et `q*`, portée par **M6**, ne peut se faire qu'une fois que M5 dispose d'un solveur qui fonctionne.
- La rédaction de la section validation, par **M6**, attend des résultats numériques concrets produits par M5.
- Le **rapport final** ne peut être assemblé que lorsque toutes les sections théoriques individuelles sont rédigées et relues par au moins une deuxième personne.

10.5 Livrables : à quoi reconnaît-on que c'est fini

Un livrable n'est pas terminé simplement parce qu'il existe. Voici, pour chaque grand livrable du projet, le critère concret qui permet de dire « c'est vraiment fini ».

Livrable	C'est terminé quand...
Graphe du réseau	il est connexe, ou les zones non connexes sont identifiées et justifiées ; il est documenté et validé par tout le groupe, pas seulement par M1.
Matrice d'incidence A	elle est construite dans le code, son rang et son conditionnement sont calculés, et le lien entre structure du réseau et ces valeurs est expliqué par écrit.
Modèle de demande	la loi choisie est justifiée, pas seulement affirmée ; les paramètres μ_i et σ_i sont calibrés et un intervalle de confiance a été calculé.
Convexité de $J(q)$	la démonstration est écrite noir sur blanc dans le rapport, pas seulement « vérifiée numériquement ».
Gradient $\nabla J(q)$	il est dérivé entièrement à la main, et relu par au moins un autre membre que celui qui l'a produit.
Solveur d'optimisation	il tourne sur un petit cas de test où la solution est connue ou vérifiable à la main, avant d'être utilisé sur le cas complet.
Une expérience numérique	le graphique et les métriques attendues sont produits, et le résultat est interprété par rapport à la théorie, pas seulement affiché.
Comparaison à la stratégie actuelle	le tableau chiffré existe pour plusieurs scénarios, pas uniquement pour la demande moyenne.
Rapport final	toutes les sections obligatoires sont présentes, aucun extrait de code source n'y figure, et la checklist de conformité (section 13) a été cochée intégralement.
Outil final	il s'exécute de bout en bout, du fichier de réseau jusqu'au graphique de comparaison, sans intervention manuelle entre les étapes.

11 Ordre chronologique du travail

Phase	Contenu
Phase 0. Cadrage (1 à 2 jours)	Tout le groupe lit le sujet et la grille d'évaluation, valide la répartition des rôles, met en place le dépôt de code et l'architecture de dossiers.
Phase 1. Fondations (en parallèle)	M1 fixe la topologie du réseau (bloquant pour M2). M3 démarre la recherche sur la loi normale et les estimateurs, sans attendre le réseau final. M4 démarre la recherche théorique sur la pénalisation et la convexité. M5 prépare l'environnement de code et l'architecture des modules. M6 commence la rédaction de l'introduction et de la revue des approches existantes.
Phase 2. Formalisation	M2 construit A dès que le graphe de M1 est validé. M3 finalise la calibration du modèle de demande. M4 dérive $J(q)$, prouve sa convexité, dérive $\nabla J(q)$ (peut avancer en parallèle de M2, mais a besoin de son conditionnement pour finaliser la borne sur le pas). M1 et M6 rédigent la section « Formulation du problème ».
Phase 3. Implémentation	M5 implémente Monte-Carlo (bloqué par M3) et le solveur d'optimisation (bloqué par M4 et M2). M2 et M4 rédigent leurs sections respectives du rapport pendant que M5 code.
Phase 4. Expérimentation	M5 exécute les 6 expériences (bloqué par un solveur fonctionnel et validé). M6 commence l'analyse des résultats dès les premiers résultats disponibles, en parallèle de M5. M3 rédige la section « Analyse statistique » en s'appuyant sur les résultats de l'Expérience 6.
Phase 5. Intégration et rédaction finale	M6 assemble le rapport complet, vérifie la conformité aux contraintes méthodologiques AMA ; tous les membres relisent leur section respective.
Phase 6. Finalisation	Répétition de la présentation/démonstration, dernières corrections, soumission.

12 Dépendances et bloqueurs

Référence rapide, à consulter en cas de doute sur ce qui peut démarrer ou non.

- Impossible de construire A avant que M1 ait figé V , E (réservoirs, quartiers, conduites).
- Impossible de dériver correctement $\nabla J(q)$ avant que la formulation pénalisée soit validée par le groupe.
- Impossible de fixer une borne théorique sûre sur le pas d'apprentissage avant d'avoir le conditionnement calculé par M2.
- Impossible de lancer Monte-Carlo avant que M3 ait figé le modèle probabiliste (μ_i , σ_i , corrélations).
- Impossible de coder le solveur d'optimisation avant que les dérivations de M4 soient validées : le code ne précède jamais la théorie.
- Impossible de comparer distribution actuelle et q^* avant d'avoir un solveur fonctionnel.
- Impossible de rédiger la section validation avant d'avoir des résultats numériques produits par M5.
- Impossible de finaliser le rapport avant que toutes les sections théoriques individuelles soient rédigées et relues.

13 Checklist finale

À parcourir collectivement avant la remise. Chaque case doit pouvoir être cochée par une personne autre que celle qui a produit l'élément correspondant.

Théorie et mathématiques

- ☐ Interprétation physique de $Aq=b$ rédigée et validée
- ☐ Rang et conditionnement de A calculés et interprétés
- ☐ Choix de la loi normale pour D_i justifié
- ☐ Estimateurs de μ_i , σ_i et intervalles de confiance établis
- ☐ Convexité de $J(q)$ démontrée
- ☐ $\nabla J(q)$ dérivé complètement et relu par au moins deux personnes
- ☐ Règle de mise à jour et projection écrites explicitement
- ☐ Conditions de convergence (pas, critère d'arrêt) établies théoriquement
- ☐ Influence de μ discutée théoriquement

Données

- ☐ Réseau synthétique construit et documenté
- ☐ Paramètres de demande (μ_i , σ_i) calibrés et justifiés
- ☐ Corrélations entre quartiers définies, si retenues

Simulation et code

- ☐ Graphe et matrice d'incidence codés et testés
- ☐ Générateur Monte-Carlo fonctionnel
- ☐ Solveur d'optimisation (gradient projeté) fonctionnel et validé sur un petit cas test
- ☐ Distribution de référence (proportionnelle) codée

Validation

- ☐ Au moins deux résultats théoriques vérifiés empiriquement (convergence, conditionnement)
- ☐ Analyse de sensibilité d'au moins un hyperparamètre réalisée

Expériences

- ☐ Les 6 expériences de la section 6 exécutées
- ☐ Figures et tableaux produits pour chacune

Comparaison

- ☐ Comparaison chiffrée distribution actuelle contre q^* sur plusieurs scénarios

Rapport

- ☐ Structure conforme (introduction, formulation, développement mathématique en 5 volets, méthode numérique en pseudo-code, expériences, analyse, limites, conclusion, bibliographie)
- ☐ Aucun extrait de code source dans le rapport
- ☐ Les cinq volets du développement mathématique sont bien présents (algèbre, probabilités, statistiques, optimisation avec gradient complet, structure discrète)

Bibliographie

- ☐ Toutes les sources utilisées sont référencées

Présentation / démonstration

- ☐ Support de présentation prêt
- ☐ Outil final exécutable de bout en bout testé avant la démonstration

14 Risques et erreurs à éviter

Commencer à coder une formule (gradient, seuil, critère d'arrêt) avant de l'avoir dérivée à la main dans le rapport : c'est une violation directe de la Contrainte méthodologique 1.

Utiliser des multiplicateurs de Lagrange au lieu de la pénalisation : c'est explicitement interdit par la Contrainte méthodologique 2.

Oublier la projection à chaque itération de la descente de gradient, et ne l'appliquer qu'à la fin : cela change fondamentalement l'algorithme.

Se tromper dans la dérivation du gradient du terme de pénalisation (facteur 2 oublié, mauvaise transposition de A) : à faire vérifier systématiquement par une deuxième personne.

Choisir un pas d'apprentissage arbitraire sans le relier à la borne théorique établie par M4 et M2, ce qui rend l'analyse de convergence peu crédible.

Se contenter d'une courbe qui descend comme preuve de convergence, sans la relier explicitement à la constante de Lipschitz et à la borne théorique sur le pas.

Sous-dimensionner le nombre de tirages Monte-Carlo, ce qui rend les statistiques peu fiables et l'analyse de robustesse peu convaincante.

Repousser la rédaction du rapport à la toute fin : la personne qui rédige (M6) doit comprendre toutes les parties, il faut donc des points de synchronisation réguliers, pas une rédaction de dernière minute.

Inclure du code source dans le rapport, même sous forme de capture d'écran : c'est interdit.

Oublier l'analyse de sensibilité obligatoire sur au moins un hyperparamètre.

Oublier la comparaison explicite à la stratégie de référence, qui est le livrable central attendu par le sujet.

Négliger la justification du choix de la loi normale pour la demande : c'est une hypothèse de modélisation à défendre, pas une évidence.

15 Plan de travail pour les premiers jours

Jour 1	Réunion de cadrage collective : lecture du sujet et de la grille d'évaluation, validation de la répartition des rôles, création du dépôt de code avec l'architecture proposée en section 7.
Jour 2	Recherche bibliographique individuelle selon la liste de la section 3. M1 propose un premier schéma de réseau (nombre de quartiers, réservoirs, conduites) à valider par le groupe.
Jour 3	Validation collective du réseau. M2 démarre la construction algébrique de A . M4 démarre la dérivation de $J(q)$ et sa preuve de convexité. M3 démarre la calibration du modèle de demande.
Jour 4-5	M4 finalise la dérivation complète de $\nabla J(q)$, la règle de mise à jour, la projection et les conditions de convergence. M2 finalise l'étude du rang et du conditionnement. M3 finalise les estimateurs et les intervalles de confiance. M1 et M6 rédigent la section « Formulation du problème ».
Jour 6-7	Point de synchronisation obligatoire : revue collective de toutes les dérivations mathématiques avant que quiconque ne commence à coder les fonctions correspondantes. C'est le jalon le plus important du projet. Une fois ce point validé, M5 démarre l'implémentation.

16 Prérequis avant de commencer l'implémentation

Ce que nous devons absolument avoir terminé avant d'écrire la première ligne de code d'optimisation.

1. Le graphe du réseau est figé (nœuds, arêtes, capacités, orientations) et validé par le groupe.
2. La matrice d'incidence A est construite formellement, son rang et son conditionnement sont étudiés.
3. Le modèle probabiliste de la demande (loi, μ_i , σ_i , justification) est validé.
4. La convexité de $J(q) = \sum c_e q_e^2 + \mu \|Aq - b\|^2$ est démontrée par écrit.
5. Le gradient $\nabla J(q)$ est entièrement dérivé à la main et relu par au moins deux membres différents de celui qui l'a produit.
6. La règle de mise à jour et la projection sur $q \geq 0$ sont écrites explicitement, sous forme de pseudo-code.
7. Les conditions de convergence (borne sur le pas, critère d'arrêt) sont déterminées théoriquement à partir du conditionnement calculé par Membre 2.
8. La définition précise de la stratégie de référence (distribution actuelle proportionnelle à la demande) est fixée.

Jalon de passage obligatoire

Tant que ces huit points ne sont pas cochés, aucune ligne de code des modules `optimization/` ne devrait être écrite.